

Assignment 2: Process Mining and Automation

Alpha-algorithm computation, Petri-net generation, and Python automation

Submission focus

This report explains the Alpha algorithm, shows the exact computations from the event log, maps the mined relations onto a Petri net, and documents a reusable Python automation workflow.

Team Members

Name	Code
Seif Eldeen Ibrahim	22P0023
Anas Bayoumi	22P0011
Nadine Radwan	22P0214
Jana Sameh	22P0237

Contents

List of Figures	ii
1 Executive Scope	1
2 Evaluation Criteria Coverage	1
3 Lecture Concepts and Algorithm	2
4 Preprocessing	2
5 Generated Petri-Net Conversion Evidence	3
5.1 Step 1: Get Process Instances	3
5.2 Step 2: Project Each Instance	3
5.3 Step 3: Aggregate Projected Relations	3
5.4 Step 4: Build Alpha Relations and Places	4
5.5 Step 5: Map Onto Petri Net	5
5.6 Timing Metrics for Interpretation	5
6 Mined Process Interpretation	6
7 Generated Petri Net	6
8 Automation Design	9
8.1 Reusable Input Contract	9
9 Timing Findings and Improvements	11
10 Conclusion	11

List of Figures

1	Alpha-mined Petri net generated from the event log.	8
2	Automation run screenshot showing the Python script and generated output files. .	10

1 Executive Scope

The assignment has two required tasks. Task One mines the provided event log using the Alpha algorithm and converts the discovered relations into a Petri net. Task Two automates the same steps from a structured CSV file. The implementation uses Python because it can parse timestamps, group cases, compute relations exactly, generate Petri-net source files, and rebuild the report tables without manual spreadsheet copying.

2 Evaluation Criteria Coverage

The assignment rubric is used as an explicit checklist. This submission applies the lecture process-mining material by generating a Petri net from the Alpha algorithm.

Criteria category	Marks	How this report covers it
Alpha algorithm application	15	The report states the Alpha definitions, builds T_W , T_I , T_O , direct succession, causality, parallelism, choice, candidate places, maximal places, source/sink places, and exact counts. Intermediate relations are included in generated tables.
Petri-net process model	15	The mined Alpha places are translated into a Petri net with source and sink places, one transition per activity, and arcs from places to transitions and transitions to places. DOT, PNML, and SVG outputs are included.
Process understanding and interpretation	10	The interpretation discusses stock/manufacturing choices, convergence at confirmation, invoice/address concurrency, payment/shipment concurrency, archive readiness, and the Case 3 event-ID anomaly.
Clarity of documentation	10	The report includes a cover page, contents, list of figures, preprocessing explanation, exact computation tables, generated Petri-net evidence, command evidence, and clear rerun instructions.
Tool selection and justification	5	Python is justified because it supports CSV import, timestamp parsing, grouping by case, deterministic relation computation, generated evidence files, and reuse on new logs.
Data preprocessing	10	The event log is structured into <code>case_id</code> , <code>timestamp</code> , <code>activity</code> , and <code>resource</code> ; events are sorted by timestamp per case; the Case 3 label/prefix inconsistency is documented and handled.
Alpha algorithm automation	15	<code>tools/process_mining_alpha.py</code> implements the Alpha logic and outputs traces, relation counts, maximal places, Petri-net DOT/PNML/SVG, timing metrics, CSV, and text summaries.
Generation of Petri net	10	The automation writes <code>output/assignment2_petri_net.dot</code> , <code>output/assignment2_petri_net.pnml</code> , and <code>output/assignment2_petri_net.svg</code> ; the visual model is validated against the same Alpha places and arcs shown in Task One.
Usability and reusability	10	The script accepts an input CSV and output directory from the command line, has a stable input schema, and can be rerun on a different event log with the same fields.

3 Lecture Concepts and Algorithm

The process-mining lecture defines the minimum event-log fields as case IDs and task IDs, with optional timestamp, resource, and data fields. This assignment uses case ID, timestamp, activity, and resource.

The Alpha relations used in the computation are:

- Direct succession: $x > y$ when x is directly followed by y in at least one case.
- Causality: $x \rightarrow y$ when $x > y$ and not $y > x$.
- Parallelism: $x \parallel y$ when both $x > y$ and $y > x$.
- Choice: $x \# y$ when neither $x > y$ nor $y > x$.

The report follows the lecture conversion sequence:

1. Get process instances by grouping the log by case and sorting every case by timestamp.
2. Project each instance into directly-following pairs.
3. Aggregate the projected pairs across all instances.
4. Build the Alpha relation sets and maximal pairs (A, B) .
5. Map the maximal pairs onto a Petri net.

In the Petri-net mapping, each activity becomes a transition. A source place feeds every start transition in T_I , a sink place receives every end transition in T_O , and every maximal Alpha pair (A, B) becomes a place with input arcs from all transitions in A and output arcs to all transitions in B .

4 Preprocessing

The raw assignment text was converted into `data/event_log.csv`, one row per event. Events are sorted by timestamp inside each case. One naming inconsistency in Case 3 was handled by trusting the activity label instead of the event ID prefix: the event ID beginning with **Sh** is labeled “Get shipping address”, and the event ID beginning with **Ge** is labeled “Ship product”. This preserves the activity sequence stated in the assignment text.

Preprocessing item	Handling
Case identification	Events are grouped by <code>case_id</code> ; four cases are present.
Timestamp handling	Timestamps are parsed with the fixed format <code>YYYY-MM-DD HH:MM</code> and sorted inside each case before mining.
Activity labels	Full activity labels are preserved for computation; shorter names are used only for report readability and diagrams.
Resource field	Resources such as <code>SYS1</code> , <code>SYS2</code> , <code>DMS</code> , and employees are retained in the cleaned CSV for handoff analysis.
Quality issue	Case 3 has two event IDs whose prefixes do not match their labels; the activity labels are treated as authoritative because the assignment text defines the actual task sequence.

5 Generated Petri-Net Conversion Evidence

5.1 Step 1: Get Process Instances

The event log is grouped by case ID and sorted by timestamp. Each case becomes one ordered process instance.

Case	Process instance trace
1	Check stock → Retrieve item → Confirm order → Get address → Emit invoice → Receive payment → Ship product → Archive order
2	Check stock → Check materials → Request materials → Obtain materials → Manufacture → Confirm order → Emit invoice → Get address → Ship product → Receive payment → Archive order
3	Check stock → Check materials → Manufacture → Confirm order → Emit invoice → Get address → Ship product → Receive payment → Archive order
4	Check stock → Retrieve item → Confirm order → Get address → Emit invoice → Receive payment → Ship product → Archive order

5.2 Step 2: Project Each Instance

Each instance is projected to the directly-following activity pairs observed inside that single trace.

Case	Projected direct-succession pairs
1	Check stock > Retrieve item; Retrieve item > Confirm order; Confirm order > Get address; Get address > Emit invoice; Emit invoice > Receive payment; Receive payment > Ship product; Ship product > Archive order
2	Check stock > Check materials; Check materials > Request materials; Request materials > Obtain materials; Obtain materials > Manufacture; Manufacture > Confirm order; Confirm order > Emit invoice; Emit invoice > Get address; Get address > Ship product; Ship product > Receive payment; Receive payment > Archive order
3	Check stock > Check materials; Check materials > Manufacture; Manufacture > Confirm order; Confirm order > Emit invoice; Emit invoice > Get address; Get address > Ship product; Ship product > Receive payment; Receive payment > Archive order
4	Check stock > Retrieve item; Retrieve item > Confirm order; Confirm order > Get address; Get address > Emit invoice; Emit invoice > Receive payment; Receive payment > Ship product; Ship product > Archive order

5.3 Step 3: Aggregate Projected Relations

The projected pairs are aggregated across all instances. The relation column is then derived using the Alpha definitions.

From	To	Count	Alpha relation
Check materials	Manufacture	1	→
Check materials	Request materials	1	→
Check stock	Check materials	2	→
Check stock	Retrieve item	2	→
Confirm order	Emit invoice	2	→
Confirm order	Get address	2	→
Emit invoice	Get address	2	
Emit invoice	Receive payment	2	→
Get address	Emit invoice	2	

Get address	Ship product	2	→
Manufacture	Confirm order	2	→
Obtain materials	Manufacture	1	→
Receive payment	Archive order	2	→
Receive payment	Ship product	2	
Request materials	Obtain materials	1	→
Retrieve item	Confirm order	2	→
Ship product	Archive order	2	→
Ship product	Receive payment	2	

5.4 Step 4: Build Alpha Relations and Places

Computed item	Value
Cases in the log	4
Events in the log	36
Activities in T_W	12
Start activities in T_I	1
End activities in T_O	1
Observed direct-succession pairs $>$	18
Causal pairs $\rightarrow$	14
Parallel ordered pairs $ $	4
Parallel unordered pairs	2
Choice ordered pairs $\#$	100
Maximal Alpha places Y_W	11
Average cycle time	5.28 days

Choice is counted as an ordered Alpha relation: $x\#y$ and $y\#x$ are both included when two activities never directly follow one another.

- T_W activities: Archive order, Check materials, Check stock, Confirm order, Emit invoice, Get address, Manufacture, Obtain materials, Receive payment, Request materials, Retrieve item, Ship product
- T_I start activities: Check stock
- T_O end activities: Archive order
- Parallel pairs: (Emit invoice, Get address), (Receive payment, Ship product)

Input activity set A	Output activity set B
{Check materials}	{Manufacture, Request materials}
{Check materials, Obtain materials}	{Manufacture}
{Check stock}	{Check materials, Retrieve item}
{Confirm order}	{Emit invoice}
{Confirm order}	{Get address}
{Emit invoice}	{Receive payment}
{Get address}	{Ship product}
{Manufacture, Retrieve item}	{Confirm order}
{Receive payment}	{Archive order}

{Request materials}	{Obtain materials}
{Ship product}	{Archive order}

5.5 Step 5: Map Onto Petri Net

Each activity becomes a transition. The source place feeds the start transition. Each maximal Alpha pair becomes a place between its input and output transitions. The sink place receives the end transition.

Place	Input transitions	Output transitions
Start	$\emptyset$	{Check stock}
p1	{Check materials}	{Manufacture, Request materials}
p2	{Check materials, Obtain materials}	{Manufacture}
p3	{Check stock}	{Check materials, Retrieve item}
p4	{Confirm order}	{Emit invoice}
p5	{Confirm order}	{Get address}
p6	{Emit invoice}	{Receive payment}
p7	{Get address}	{Ship product}
p8	{Manufacture, Retrieve item}	{Confirm order}
p9	{Receive payment}	{Archive order}
p10	{Request materials}	{Obtain materials}
p11	{Ship product}	{Archive order}
End	{Archive order}	$\emptyset$

Petri-net element	Count
Places, including source and sink	13
Transitions	12
Arcs	28

5.6 Timing Metrics for Interpretation

Case	Start	End	Cycle time
1	2012-07-30 11:14	2012-08-06 09:12	6.92 days
2	2012-08-01 09:44	2012-08-07 08:50	5.96 days
3	2012-08-02 10:06	2012-08-06 10:25	4.01 days
4	2012-08-04 08:11	2012-08-08 14:08	4.25 days

From	To	Occurrences	Average hours
Ship product	Receive payment	2	76.28
Emit invoice	Receive payment	2	57.32
Get address	Emit invoice	2	46.76
Request materials	Obtain materials	1	44.63
Obtain materials	Manufacture	1	30.88
Receive payment	Ship product	2	14.42
Ship product	Archive order	2	11.22

Check materials	Manufacture	1	5.88
Check stock	Retrieve item	2	3.46
Check materials	Request materials	1	1.10
Get address	Ship product	2	1.07
Retrieve item	Confirm order	2	0.47
Confirm order	Emit invoice	2	0.42
Confirm order	Get address	2	0.32
Check stock	Check materials	2	0.22
Emit invoice	Get address	2	0.20
Manufacture	Confirm order	2	0.13
Receive payment	Archive order	2	0.12

6 Mined Process Interpretation

The mined process has one start activity, *Check stock availability*, and one end activity, *Archive order*. The exact computations show 4 cases, 36 events, 12 activities, 18 observed direct-succession pairs, 14 causal pairs, 2 unordered parallel pairs, and 11 maximal Alpha places. The generated Petri net contains 13 places including source and sink, 12 transitions, and 28 arcs.

The behavior discovered from the traces is:

- After stock checking, the process chooses between retrieving an available warehouse item and entering the materials/manufacturing branch.
- In the manufacturing branch, checking materials may lead directly to manufacturing or to requesting and obtaining raw materials first.
- Warehouse retrieval and manufacturing converge at confirming the order.
- Invoice/address order differs across traces, so the Alpha algorithm detects *Emit invoice* and *Get shipping address* as parallel.
- Payment/shipment order also differs across traces, so *Receive payment* and *Ship product* are parallel.
- Archiving is reached after the payment and shipment branches provide archive-ready evidence in the log.

The log does not show a repeated-activity loop inside a case, so the mined model focuses on choices and concurrency rather than loop behavior. The main anomaly is the Case 3 event-ID prefix mismatch documented in preprocessing; it is treated as a data-quality issue rather than a behavioral exception because the activity names remain consistent with the overall process.

7 Generated Petri Net

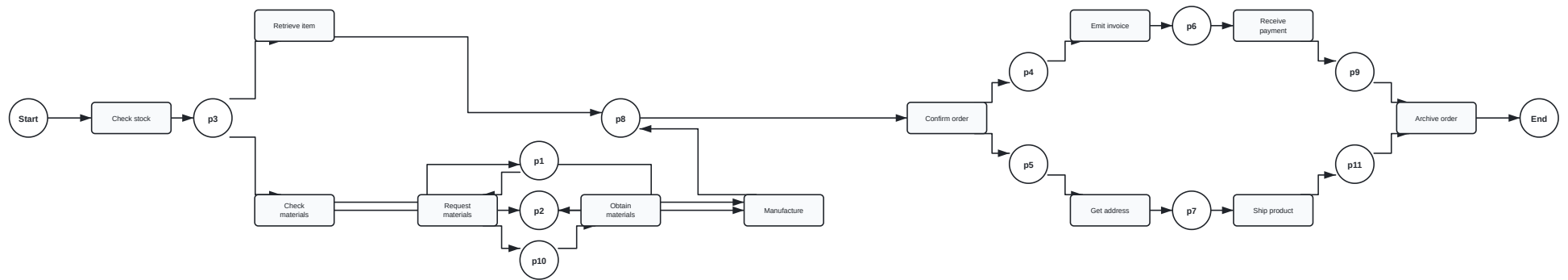
The generated process model is stored in standard and review-friendly formats:

- `output/assignment2_petri_net.dot`
- `output/assignment2_petri_net.pnml`
- `output/assignment2_petri_net.svg`

The source place connects to *Check stock*. The *Check stock* transition leads to a place that enables either *Retrieve item* or *Check materials*. The manufacturing branch includes places for requesting

and obtaining raw materials when required. *Manufacture* and *Retrieve item* converge before *Confirm order*. After confirmation, separate places enable the invoice and address transitions. Later places model payment and shipment behavior before *Archive order* reaches the sink place.

Assignment 2 Alpha-Mined Petri Net



8 Automation Design

The automation script is `tools/process_mining_alpha.py`. It performs the following reusable workflow:

1. Read any CSV containing `case_id`, `timestamp`, `activity`, and `resource`.
2. Sort events per case by timestamp.
3. Get process instances from the sorted cases.
4. Project each instance into directly-following pairs.
5. Aggregate the projected pairs and derive causality, parallelism, and choice using the lecture definitions.
6. Build maximal Alpha places (A, B) .
7. Map the places and activities onto a Petri net.
8. Generate documentation outputs and reusable visualization files.

Run command:

```
python3 tools/process_mining_alpha.py \
  --input data/event_log.csv \
  --output-dir output
```

Generated file	Purpose
output/event_log_sorted.csv	Clean sorted event log for checking preprocessing.
output/assignment2_generated_tables.tex	Exact Alpha and Petri-net conversion evidence tables included in this report.
output/assignment2_petri_net.dot	Graphviz source for the generated Petri net.
output/assignment2_petri_net.pnml	PNML source for Petri-net review or import into Petri-net tools.
output/assignment2_petri_net.svg	SVG Petri-net diagram rendered into this report after validation.
output/assignment2_summary.txt	Plain-text summary of counts, relations, places, transitions, and arcs.

8.1 Reusable Input Contract

Reuse requirement	Implementation
Input schema	Any CSV can be used if it contains <code>case_id</code> , <code>timestamp</code> , <code>activity</code> , and <code>resource</code> . An optional <code>event_id</code> column is read when present.
Configurable paths	The command-line options <code>-input</code> and <code>-output-dir</code> let the user run the script on another log without changing the code.
Transparent outputs	The script writes the sorted log, generated LaTeX tables, Petri-net DOT, Petri-net PNML, Petri-net SVG, and text summary so intermediate steps can be checked.
Validation method	Counts in the summary must match the generated report tables; the Petri-net places and arcs must match the Step 5 mapping table.

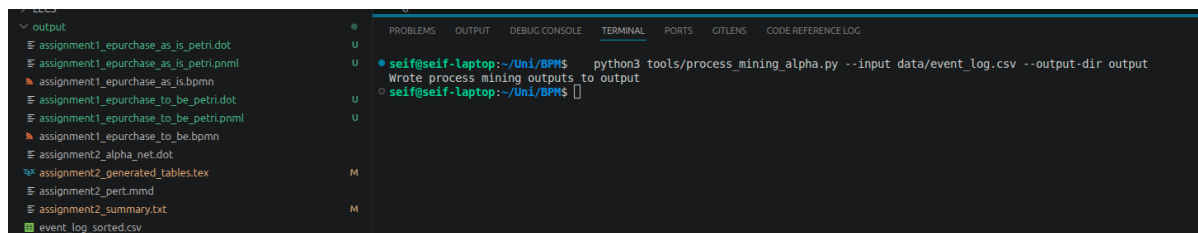


Figure 2. Automation run screenshot showing the Python script and generated output files.

9 Timing Findings and Improvements

Timing metrics are used only to interpret bottlenecks in the event log; they are not used to define the Petri-net structure. The average observed cycle time is 5.28 days. The longest average waits are *Ship product* to *Receive payment* at 76.28 hours, *Emit invoice* to *Receive payment* at 57.32 hours, and *Get shipping address* to *Emit invoice* at 46.76 hours. These delays are waiting periods between handoffs, not the short system tasks themselves.

Recommended improvements are:

- Integrate payment and shipment status so archive readiness is visible immediately.
- Add alerts for long gaps after invoice emission or shipment dispatch.
- Standardize whether payment must precede shipment or whether both may proceed independently; the log shows both orders.
- Monitor resource handoffs between SYS1, SYS2, employees, and DMS because handoffs are common sources of delay and error.

10 Conclusion

The Alpha algorithm discovers an order-fulfillment process with stock/manufacturing choices and two concurrent sections. The automation produces the same evidence needed for Task One and reusable files for Task Two. The included Petri-net SVG and automation screenshot document the generated process model and script execution evidence.